Full JS Test

Hello World

ashishbindra2@gmail.com Switch account

* Indicates required question

Email *

Record ashishbindra2@gmail.com as the email to be included with my response

Request edit access

What would be the output of the following code snippet? * 1 point

```
console.log(1);

setTimeout(() => console.log(2));

Promise.resolve().then(() => console.log(3));

Promise.resolve().then(() => setTimeout(() => console.log(4)));

Promise.resolve().then(() => console.log(5));

setTimeout(() => console.log(6));

console.log(7);
```

- 1 2 3 4 5 6 7
- 1 7 3 5 4 2 6
- 1 7 3 5 2 6 4
- 1 2 6 4 3 5
- 1 2 6 3 4 5

Request edit access

This is the first question from a series of questions based on Promise functionality. * 1 point

```
function init() {
  throw new Error("I am an error");
  return Promise.resolve(1);
}

init()
  .then((v) => console.log(v + 1))
  .catch((err) => console.log("Error caught -- ", err));
```

- ○ 2
- ○ Error caught -- I am an error
- ○ Uncaught Error - I am an error
- ○ 2 Error caught -- I am an error

Request edit access

What would be the output of the following code snippet? * 1 point

```
function processing() {
  return new Promise((resolve, reject) => {
    resolve(1);
    reject("Failed");
    resolve(2);
  });
}

function init() {
  processing()
    .then((v) => console.log(v + 1))
    .catch((err) => console.log(err));
}

init();
```

- 2, Failed
- 2, Failed ,3
- 1 , Failed , 2
- Failed
- Uncaught (in promise) Failed
- 2
- 1

Request edit access

What would be the output of the following code snippet? * 1 point

```
function Person(name) {
  this.name = name;
}
Person.prototype.getName = () => {
  return this.name;
};
const yomesh = new Person('Yomesh');
console.log(yomesh.getName());
```

- undefined
- Value of the property name on the global object
- Yomesh
- Empty String
- Error: getName is not a function
- Window

Request edit access

What is the output of the following? *

1 point

```
let items = [];

items[null] = "foo";
console.log(items.length);
items[100] = "bar";
console.log(items.length);
```

- 0 1
- 1 2
- 0 2
- 0 101
- 0 100

Request edit access

What is the output of the following code snippet? *

1 point

```
var foo = 1;
var change = () => {
  this.foo = 2;
  console.log(this.foo);
};
var obj = {
  foo: 3
};
var bounded = change.bind(obj);

// What would be the output of the following?
console.log(foo);
console.log(change());
console.log(foo);
console.log(obj.foo);
console.log(bounded());
```

- 1, 2 ,2 ,3 ,2
- 1,2,1,3,3
- 1,2,2,2,3
- 1,1,2,3,1

Request edit access

If asked to glance over the following code snippet and identify the behaviour of the duplicate function then what do you think is happening in the following code snippet?, describe the output of the code with explanation

* 1 point

```
const original = {
  id: 'xhdyt0123',
  link: 'https://www.youtube.com/watch?v=_eaCs-pzaVg',
  metadata: {
    title: 'Build Your Own Redux',
    description: 'In this video we are going to see how we an build our own Redux',
  },
  published: true
}

const duplicate = (original) => {
  if (!original.published) {
    throw new Error('Your post needs to published before duplication');
  }

  const copy = {
    id: +new Date(),
    link: original.link,
    metadata: original.metadata,
    published: original.published
  };

  copy.metadata.title = `Copy of ${original.metadata.title}`;

  return copy;
};

output = duplicate(original)
conosole.log(original , output);
```

Your answer

Request edit access

Which function is used to serialize an object into a JSON string in Javascript? * 1 point

- JSON.convert()
- JSON.string()
- JSON.stringify()
- JSON.parse()

Request edit access

What do you think would be the output of the following code snippet? * 1 point

```
class Person {
        constructor(name) {
                this.name = name;
        }

        print = () => {
                console.log(this.name);
        }
};

class Employee extends Person {
        constructor(name, id) {
                super(name);
                this.id = id;
        }

        print() {
                console.log(this.name, this.id);
        }
};

const one = new Person('one');
one.print();

const two = new Employee('two', 2);
two.print();
```

- one , two
- one, two, 2
- one, undefined, undefined
- one, undefined, 2
- Uncaught Error

Request edit access

What will be the output of the following code? * 1 point

```
Promise.reject('Error!') .then(() => 99, err => 42)
.then(console.log);
```

- 'Error!'
- 99
- 42
- Nothing is logged

Promise.resolve(1).then(2).then(console.log); * 1 point

```
Promise.reject(1).then(2, 2).then(console.log, console.log);
```

- 2 , 2
- Promise <Pending>
- 1 ,1
- Error
- Nothing will Log

Request edit access

What will be the output of the following code snippet? * 1 point

```
async function asyncFunc() { return 10; }
console.log(asyncFunc());
```

- 10
- Promise {<fulfilled>: 10}
- [object Promise]
- undefined

What will the following code output? * 1 point

```
Promise.resolve()
  .then(() => {
    throw new Error('Whoops!');
  })
  .catch(err => console.log(err.message))
  .then(() => console.log("Next success"))
  .catch(err => console.log("Next error"));
```

- Whoops!
- Whoops! followed by Next error
- Whoops! followed by Next success
- Next success
- Option 5

Request edit access

Which of the following is true about the static method Promise.allSettled()? * 1 point

- It returns a promise that resolves after all of the given promises have either resolved or rejected, with an array of objects describing the outcome of each promise.
- It behaves exactly like Promise.all() but doesn't short-circuit when a promise is rejected.
- It is not a valid Promise method.
- A and B

```
Promise.resolve(5)
  .then(x => x * 2)
  .then(x => Promise.resolve(x * 2))
  .then(console.log);
```

* 1 point

- 10
- 20
- Promise {<Fullfilled> :20}
- 5

Request edit access

const obj = { 1: 'one', 2: 'two', 3: 'three', length: 3, push: Array.prototype.push, }; obj.push('four'); console.log(obj.length); * 1 point

- 3
- 4
- undefined
- TypeError

typeof typeof 1 * 1 point

- 'number'
- 'string'
- 'undefined'
- 'object'

console.log(+'42px'); * 1 point

- 42
- '42px'
- NaN
- 'NaNpx'

Request edit access

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Promise 1 resolved"), 1000);
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Promise 2 resolved"), 500);
});

Promise.all([promise1, promise2]).then((values) => {
  console.log(values);
});
```

*

1 point

- ["Promise 1 resolved", "Promise 2 resolved"]
- ["Promise 2 resolved", "Promise 1 resolved"]
- Error
- Nothing

Request edit access

```
function foo() {
  return new Promise(resolve => {
    resolve('foo');
  });
}

async function bar() {
  return 'bar';
}

console.log(foo() instanceof Promise);
console.log(bar() instanceof Promise);
```

*

1 point

- true, true
- false, true
- true, false
- false, false

(function () { console.log(typeof arguments); })(); *

1 point

- 'object'
- 'array'
- 'undefined'
- 'function'

Request edit access

What does the Object.keys() method return? * 1 point

- An array of all enumerable properties of an object
- An array of all properties (enumerable or not) of an object
- An array of all own properties (enumerable or not) of an object
- An array of all own enumerable properties of an object

What is the purpose of the hasOwnProperty() method in JavaScript? * 1 point

- Checks if a property exists in an object
- Adds a new property to an object
- Removes a property from an object
- Iterates over the properties of an object

console.log(typeof NaN); * 1 point

- 'number'
- 'NaN'
- 'undefined'
- 'string'

Request edit access

What is the purpose of the Array.prototype.filter() method in JavaScript? * 1 point

- Modifies the original array
- Removes elements from an array based on a condition
- Sorts the elements of an array
- Converts elements of an array into a string

```
const arr = [1, 2, 3];
arr.forEach((num, index, array) => {
  array[index] = num * 2;
});
console.log(arr);
```

* 1 point

- [2, 4, 6]
- [1, 2, 3]
- [2, 4, 6, 8]
- [2, 4, 6, 2, 4, 6]

Request edit access

Which method is used to check if a value is an array in JavaScript? * 1 point

- isArray()
- isTypeOf()
- typeOf()
- arrayType()

console.log(1 + '2' + '2'); * 1 point

- 122
- 15
- 32
- NaN

2 == [[[2]]] * 1 point

- true
- false
- NaN
- TypeError

Request edit access

```
function outer() {
  const x = 10;
  function inner() {
    console.log(this.x);
  }
  inner();
}
outer.call({ x: 20 });
```

*

1 point

- 20
- undefined
- Error
- 10

const arr = [1, , 3, 4, , 6]; console.log(arr.with(0, 2)); *

1 point

- [2, undefined, 3, 4, undefined, 6]
- [2, undefined,undefined,undefined, 3,4,5]
- [1,0,2,3,4,5,6]
- [1,[2 , 3] , 3, 4, [5] , 6]

Request edit access

console.log([1, , 3].some((x) => x === undefined)); *
console.log([1, , 1].some((x) => x !== 1));
console.log([1, undefined, 1].some((x) => x !== 1));

1 point

- false, true. false
- true, false, true
- true, true, false
- false, true,false
- false, false, true

const obj1 = { internal: {}, }; Object.freeze(obj1);
obj1.internal.a = "aValue"; obj1.internal.a;

* 1 point

- Reference Error
- not accessable due to non-enumerable
- aValue
- undefined

Request edit access

what is output? *

1 point

```
class Chameleon {
  static colorChange(newColor) {
    this.newColor = newColor;
    return this.newColor;
  }

  constructor({ newColor = 'green' } = {}) {
    this.newColor = newColor;
  }
}

const freddie = new Chameleon({ newColor: 'purple' });
console.log(freddie.colorChange('orange'));
```

- TypeError
- green
- orange
- purple

Request edit access

```
function getAge() { *
  'use strict';
  age = 21;
  console.log(age);
}

getAge();
```

1 point

- 21
- undefined
- ReferenceError
- TypeError

```
const obj = { 1: 'a', 2: 'b', 3: 'c' }; *
const set = new Set([1, 2, 3, 4, 5]); obj.hasOwnProperty('1');
obj.hasOwnProperty(1);
set.has('1');
set.has(1);
```

1 point

- false true false true
- false true true true
- true true false true
- true true true true

A copy of your responses will be emailed to ashishbindra2@gmail.com.

Request edit access

Submit

Clear form

Never submit passwords through Google Forms.

This content is neither created nor endorsed by Google. Report Abuse - Terms of Service - Privacy Policy

Google Forms

Request edit access

Request edit access